BSCS 5th Semester | PUACP | Web Technologies – EC-331
Topic: Web Services (SOAP & REST)
By Noorayy Jam

Introduction to Web Services

Web services are standardized methods that allow applications to communicate over a network, typically the Internet. They form the backbone of modern distributed systems, powering web applications, mobile apps, cloud platforms, e-commerce systems, and social media APIs.

Web services use **open standards** like XML, JSON, SOAP, REST, WSDL, and HTTP to enable communication between heterogeneous platforms. Their primary purpose is to ensure **interoperability**, meaning systems written in different programming languages or running on different operating systems can still exchange data seamlessly.

There are two main categories of web services: 1. **SOAP (Simple Object Access Protocol)** – A protocol that defines strict message formats and communication rules. 2. **REST (Representational State Transfer)** – An architectural style that focuses on simplicity and stateless communication over HTTP.

Both SOAP and REST have distinct architectures, benefits, and limitations, which are crucial to understand when designing web-based solutions.

SOAP (Simple Object Access Protocol)

1. Introduction

SOAP is a **protocol** designed for structured information exchange in web services. It uses **XML** for message formatting, ensuring platform and language independence. Developed by Microsoft in 1998 and standardized by W3C, SOAP enables applications on different systems to communicate securely and reliably.

It is widely used in enterprise environments where reliability, transaction integrity, and security are top priorities.

2. Key Features of SOAP

- **Protocol-Based:** SOAP enforces strict rules and communication standards.
- **XML-Based:** Messages are formatted in XML, ensuring readability across all systems.
- **Extensible:** Supports extensions like WS-Security and WS-ReliableMessaging.
- **Transport Independent:** Can operate over HTTP, HTTPS, SMTP, or TCP.
- **Secure:** Supports WS-Security for encryption, integrity, and authentication.
- **Reliable:** Ensures message delivery even in unreliable networks.
- **Neutral:** Works across different programming languages and platforms.

3. SOAP Architecture

SOAP messages follow a strict structure comprising four main components: 1. **Envelope:** The root element that defines the message boundaries. 2. **Header:** Contains optional metadata such as authentication tokens or session IDs. 3. **Body:** The actual payload or data content being exchanged. 4. **Fault:** Used to report errors encountered during message processing.

SOAP Message Example:

```
<Envelope>
  <Header>
    <Authentication>
      <Username>Human</Username>
      <Password>12345</Password>
    </Authentication>
  </Header>
  <Body>
    <GetStudentDetails>
      <StudentID>101</StudentID>
    </GetStudentDetails>
  </Body>
  <Fault>
    <Code>500</Code>
    <Message>Internal Server Error</Message>
  </Fault>
</Envelope>
```

4. Advantages of SOAP

- High security through WS-Security.
- Reliable delivery using WS-ReliableMessaging.
- Operates across multiple transport protocols.
- Well-defined structure and global standards.
- Excellent error handling with SOAP Fault.
- Ideal for complex, transaction-oriented applications.

5. Disadvantages of SOAP

- Complex implementation.
- Performance overhead due to verbose XML.
- High bandwidth consumption.
- Slower compared to lightweight alternatives.

6. When to Use SOAP

- In **banking and finance** for secure transactions.
- In **healthcare systems** for reliable and compliant data exchange.

- For **enterprise software** like ERPs and CRMs.
- When **stateful operations** are required.
- When **guaranteed message delivery** is essential.

REST (Representational State Transfer)

1. Introduction

REST is an **architectural style** used to design networked applications. It primarily uses **HTTP/HTTPS** for communication and supports multiple data formats like JSON, XML, or plain text. RESTful APIs focus on simplicity, scalability, and stateless communication.

It is widely used in modern web, mobile, and cloud-based systems.

2. Key Features of REST

- **Stateless:** Each request contains all necessary information.
- **Resource-Oriented:** Data entities are represented as resources with unique URLs.
- **Standard HTTP Methods:**
- `GET` → Retrieve data
- `POST` → Create data
- `PUT/PATCH` → Update data
- `DELETE` → Remove data
- **Flexible Data Formats:** JSON, XML, or others.
- **Cacheable:** Supports caching for better performance.
- **Client-Server Separation:** Independent development of client and server.
- **Layered System:** Supports intermediaries like proxies and gateways.

3. REST Architecture

- **Resources:** Represent data entities, accessed via unique URIs.
- **Representation:** Resources are transferred as JSON, XML, or HTML.
- **Statelessness:** Each request is independent; the server stores no session info.
- **Caching:** Improves speed and reduces server load.
- **Uniform Interface:** Standard HTTP methods and status codes.

REST API Example:

```
GET /students/101       --> Retrieve student details
POST /students          --> Create a new student
PUT /students/101       --> Update student details
DELETE /students/101    --> Delete student
```

JSON Response Example:

```
{
  "id": 101,
  "name": "Human",
  "course": "Unknown",
  "semester": 0
}
```

4. Advantages of REST

- Lightweight and easy to implement.
- Faster performance due to smaller message sizes (JSON).
- Highly scalable and flexible.
- Platform-independent and widely adopted.
- Supports caching to improve efficiency.

5. Disadvantages of REST

- No built-in security; must rely on HTTPS.
- No strict standards, leading to variation in implementations.
- Stateless design makes session management harder.
- Error handling is not standardized.
- Not suitable for complex, multi-step operations.

6. When to Use REST

- For **mobile and web apps** requiring quick and efficient communication.
- In **microservices architectures**.
- For **public APIs** and **cloud integrations**.
- When speed, scalability, and simplicity are the main goals.

SOAP vs REST Comparison

Feature	SOAP	REST
Full Form	Simple Object Access Protocol	Representational State Transfer
Type	Protocol	Architectural Style
Message Format	XML only	JSON, XML, HTML, Plain text
Transport Protocol	HTTP, SMTP, TCP	HTTP only
Standards	Strict (WS-Security, WS-Addressing)	No strict standards

Feature	SOAP	REST
Performance	Slower due to XML	Faster and lightweight
Error Handling	SOAP Fault element	HTTP status codes
Security	High (WS-Security, Encryption)	Depends on HTTPS, OAuth, JWT
Flexibility	Less flexible	More flexible
Caching	Not easily cacheable	Easily cacheable
Best Use	Enterprise, Banking, Healthcare	Web, Mobile, Cloud, APIs
Interface Description	WSDL (Web Services Description Language)	OpenAPI / Swagger

Conclusion:
SOAP and REST both serve as essential technologies in modern web systems. SOAP offers robust security, reliability, and strict standardization, making it ideal for enterprise and critical systems. REST, on the other hand, provides lightweight, scalable, and flexible integration suitable for most modern web and mobile applications.